

Claw'deck (SideCrab) 小白指南



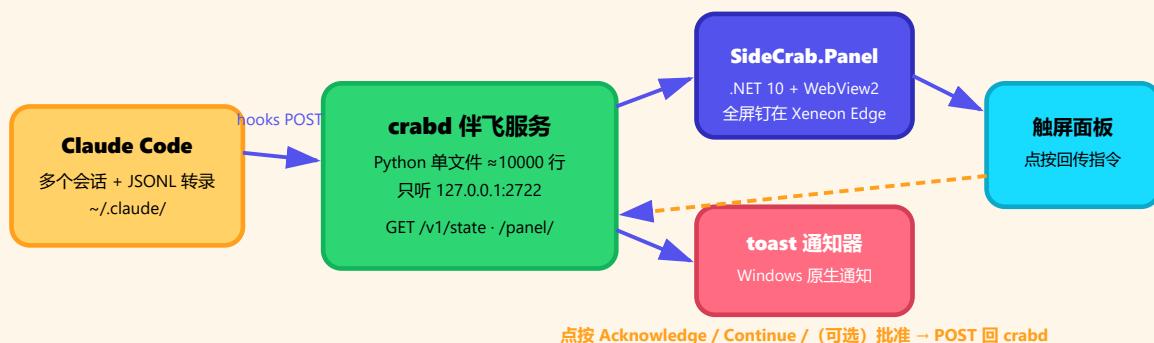
给 Claude Code 装一块“实体仪表盘”——教你读懂 2026 年爆火的桌面触屏 Agent 管理面板，并用 180 行纯标准库复刻它的软件核心

项目	SideCrab / Claw'deck — Corsair Xeneon Edge 触屏上的 Claude Code 环境状态面板
开源仓库	github.com/Dixie-sketch/Clawdeck (镜像 SideCrab 原仓库) · MIT 许可 · Python + C#/.NET 10
本仓库版本	d658d41 (SideCrab 0.34.1, companion crabd 0.37.1, 2026-10-01 仍在活跃更新)
一句话定位	螃蟹的心情就是集群状态: 多个 Claude 会话谁在干活、谁卡住等你, 一瞥即知, 还能隔着桌子点一下回答它
本指南实验	mini-crabd: 纯标准库复刻 hooks → 状态机 → /v1/state → 面板 的软件主链路 (无需 Xeneon Edge 硬件)

Q1 这到底是什么? 解决什么痛点?

你同时开着 3~5 个 Claude Code 会话干活。谁跑完了、谁在等你批准、谁已经因 API 限流挂了——以前要靠 Alt+Tab 把每个终端轮一遍。SideCrab 把这些信息放上一块摆在显示器下面的实体触屏 (约 \$250 的 Corsair Xeneon Edge, 2560×720), 配一只像素螃蟹: 螃蟹 calm = 一切安好; alert = 有会话在等你; worried = 数据馈送失联。会话提问时, 你从房间另一头就能看见, 甚至直接在屏上点"Continue"回答它。

Q2 整体架构长什么样? (三件套 + 一条数据流)



crabd (伴飞服务) 是心脏: Claude Code 的 hooks 每个事件发生时 POST 一条 JSON 给它; 它把事件汇成每会话状态, 再把全部状态装进一份 JSON 馈送 /v1/state。面板宿主加载 crabd 提供的 /panel/ 网页并整屏钉在触屏上。三件套全部只在本机 127.0.0.1 通信, 唯一的出网调用是拿你自己的令牌查一次用量限额——和 Claude Code 自己打的那个请求一模一样。

Q3 hooks: 数据是怎么“推”进来的? command 型和 http 型有何区别?

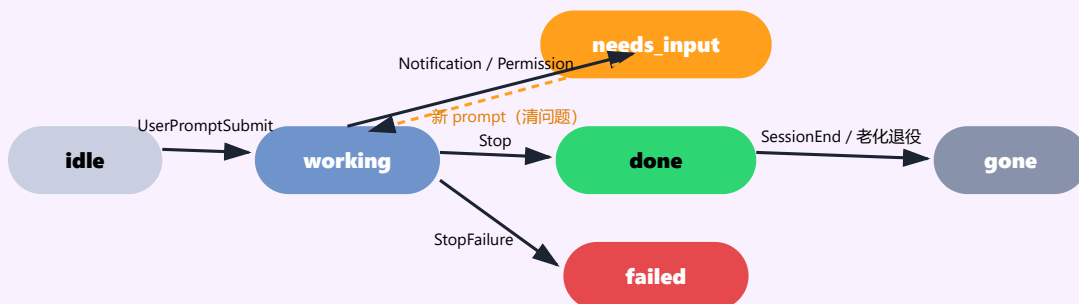
SideCrab 在 `~/.claude/settings.json` 里注册了 10 个 hook 事件: SessionStart、UserPromptSubmit、Notification、Stop、StopFailure、SubagentStart/Stop、PermissionRequest、SessionEnd、PreCompact。其中 9 个是 `"type": "http"`——由 Claude Code 本体直接发 HTTP POST, 不落 shell、不起子进程 (旧版曾用 `curl.exe` 外壳命令, v0.34 起废除); SessionStart 是例外 (CLI 在该事件不跑 http hook), 保留 command 型 curl 条目。

最关键的设计是 **fail-open (失败放行)**: hook 超时/拒连/非 2xx 时, Claude Code 得到空决策、照常继续。**crabd 挂掉的唯一后果是面板上出现一只担心的螃蟹**——你的会话永远不会被仪表盘拖死。PermissionRequest 是唯一一长超时 (60s), 因为它承担“面板批准”的长轮询。实验中我们实证了这一点: crabd 未运行时上游 statusline 脚本静默降级、退出码 0。

Q4 /v1/state: 为什么是“单一 JSON 馈送”? schema 永远是 5 是什么操作?

所有消费者 (面板页、通知器) 都只轮询/订阅这一个端点, 拿到整幅图景。一份快照 = 单一事实源, 没有“端点 A 说完了端点 B 才更新”的时序 bug。版本策略更是亮点: 文档里写明 **“schema”只在破坏性变更时升号 (钉在 5), 新增字段靠“字段存在性探测”, 未知键一律忽略**——这样 crabd 随时可以加字段而不会把已装出去的面板变砖; 只有真正的形状破坏才需要协调升级。这和知识库 [[健康探测三态]] 的思想相通: 状态必须能表达“我不知道”, 未知值渲染成破折号而非 0, 数据过期 30 秒就亮出“数据截至 HH:MM”横幅 + worried 螃蟹, 绿色永远意味着数据是新鲜的。

Q5 会话状态机: idle → working → done / failed / needs_input / gone



done/failed 停留 10 分钟 (DONE_DROP_SEC) 后退役; 重新写入转录 = 重试成功, 可回到 working

排序规则: **needs_input 最前** (最该被看见), failed 第二, working 第三。v0.36 前只有 5 种状态, API 错误挂掉的会话会伪装成“还在思考”15 分钟——**限流挂起的会话和认真干活的会话在屏上看起来一模一样**, 这就是加 failed 的理由。桌面项目读 open set: 见到陌生状态回落到中性卡片即可, 别把枚举写死。

Q6 token 燃烧量怎么算? 为什么复用 OAuth 令牌?

crabd 只读解析 `~/.claude` 里的会话 JSONL 转录 (关联 [[JSONL事件日志与折叠模型]]), 把每条 assistant 消息的 `usage.output_tokens` 累加成“今日燃烧”, 配可选的每日预算告警。限额表读的则是 Claude Code 本机那把 OAuth 令牌——同一把钥匙、同一次调用, 不新增任何授权面。坑在于该令牌约 6 小时过期、只在终端 `claude` 会话发 API 调用时刷新: 纯桌面 App 用户第二天表就会全黑。官方解法是 `claude setup-token` 换长期令牌、DPAPI 加密存放, 仅在 CLI 令牌失效时启用, 永不日志、永不下发。

Q7 面板上点"批准"真的能放行权限请求？为什么默认关闭？

能——PermissionRequest hook 长轮询 60 秒，面板点"Approve"经 crabd 把决策送回会话。但出厂**审批功能是关的**，且配对了 pairing code。背后是一次真实攻防（SC-01）：crabd 的来源门对"没有 Origin 头的请求"放行（原生客户端本就没有 Origin），于是**本机任意进程都能 POST /v1/config 改写 continue 提示词白名单，下一次 Stop hook 就会把它作为会话的下一轮指令送进模型**——这本质是知识库 [[提示注入]] 的本机变体。修复：continue 词表改回只读文件（file-only），/v1/config 拒绝三个相关键。教训：凡是"能写进模型输入的通道"，就要按最高信任级别设防，哪怕它只听 127.0.0.1。

Q8 为什么用 .NET + WebView2 宿主，而不是开一个浏览器 F11 全屏？

浏览器全屏窗口会抢焦点、有任务栏项、休眠/换屏后可能"掉"回主屏。SideCrab.Panel 是个无边框、置顶、**从不夺焦、无任务栏项**的 WebView2 窗口，按显示器 PnP 设备 id（CRXED00）钉死在 Xeneon Edge 上，休眠唤醒、拔插显示器后自动重新钉回。网页部分照常迭代（HTML/JS 由 crabd 提供），原生壳只负责"老老实实贴在那块屏上"。这是**混合栈**的典型取舍：数据面 Python（快速迭代一万行不出事）、展示壳 C#（干操作系统级的脏活）、界面本身 Web（生态最便宜）。配套工程同样值得学：升级走 `dist.staging` 试跑 → `dist.last-good` 保底 → 改名切换 → 失败自动回滚，退出码从不撒谎（关联 [[发布交换与健康门回滚]]）。

Q9 同类方案比较：这块屏凭什么火？

方案	载体	强项	局限
SideCrab / Claw'deck	Xeneon Edge 触屏（Win 专用）	跨房间可见 + 触屏应答 + 全量开源	要一块 \$250 硬件、Windows only
ClawDeck 等 Stream Deck 方案	Elgato Stream Deck 按键板	按键即操作、便宜	小屏信息量少，偏"按钮"不偏"仪表"
statusline / 终端内方案	现有终端状态栏	零硬件	必须盯着那个终端才看得见
ChibiDeck / Clawdmeter 等	ESP32 小圆屏/AMOLED	功耗低、造型萌、专注用量表	算力弱，复杂面板和双向交互吃力

共同思想都是**环境显示（ambient display）**：信息不该等你去找它，它应该一直在你余光里。SideCrab 赢在把"看"升级成了"顺手答"。

Q10 和我知识库里有概念怎么挂钩？

① **[[人机协同Interrupt]]**：SideCrab 是 Interrupt 的物理外设——"等你批准/等你输入"从终端提示变成房间级信号；② **[[零依赖编程]]**：hooks 与 statusline 脚本只用标准库、失败静默降级；③ **[[健康探测三态]]**：fresh/stale/dead 三态 honest 渲染（worried 螃蟹 + 数据截至横幅 + 未知渲染为 —）；④ **[[提示注入]]**：SC-01 是它的本机面板变体；⑤ **[[JSONL事件日志与折叠模型]]**：燃烧量的来源。新增长的轴是**"把 Agent 遥测做成实体硬件"**这个方向本身。



这个项目给了我什么能力清单

- **看懂一套完整的生产级桌面伴飞系统**：hooks 采集 → 本机服务 → 单一馈送 → 专用显示壳 → 通知回传，五环全开源。
- **hooks 工程范式可直接搬走**：http 型 hook + fail-open + 2~3s 超时上限，让"仪表盘永不拖死会话"。
- **schema 版本策略**：钉死破坏性版本号 + 字段存在性探测，桌面端与云端解耦升级。
- **敢给 Agent 系统做硬件外设了**：照着 mini-crabd 加一块 ESP32/任意屏就能复刻属于自己的仪表盘。
- **升级/回滚工程**：staging 试跑 → last-good 保底 → 原子换名，任何本地服务自更新都可套用。
- **安全直觉**：localhost ≠ 信任边界；能写入模型输入的通道按最高级别设防。

主实验：mini-crabd —— 180 行纯标准库复刻 SideCrab 软件主链路

不必买 Xeneon Edge。我们用 `exercise\mini_crabd.py`（标准库 `http.server`）复刻 **hooks** → **状态机** → `/v1/state` → `/panel/` 四环，真实运行于 `:2799`。

Step 1 · 启动伴飞服务

目的：起一个只听本机回环的状态服务。操作：

```
cd F:\reminder\clawdeck\exercise
python mini_crabd.py 2799
# → mini-crabd listening on http://127.0.0.1:2799 (panel: /panel/)
```

✅ **预期结果：**服务启动行打印，进程驻留。

Step 2 · 模拟 Claude Code 打 hook 事件（会话 s1 开跑）

目的：验证 hooks 入口与 fail-open 语义（永远 204 无正文）。操作：

```
curl -s -o NUL -w "%{http_code}" -X POST http://127.0.0.1:2799/v1/hook/session-start -d "
{"session_id\":\"s1\",\"cwd\":\"F:/reminder/clawdeck\"}"
curl -s -o NUL -w "%{http_code}" -X POST http://127.0.0.1:2799/v1/hook/prompt -d "
{"session_id\":\"s1\",\"transcript_path\":\"F:/reminder/clawdeck/exercise/sample_transcript.jsonl\"}"
curl -s http://127.0.0.1:2799/v1/state
```

✅ **预期结果：**两个 204；state 里 `s1 = working`，`burn = {"jsonlEntries": 4, "outputTokens": 440}`——JSONL 转录被真实解析（88+142+210）。

Step 3 · 让会话“开口问你”（中文问题）

目的：验证 `needs_input` 与问题捕获。中文经 `--data-binary @file` 送 UTF-8（Git Bash 内联中文会翻车，坑见下）：

```
curl -s -o NUL -w "%{http_code}" -X POST http://127.0.0.1:2799/v1/hook/notification --data-
binary @notif.json
# notif.json = {"session_id":"s1","message":"可以删除 temp 目录吗?"}
```

✅ **预期结果：**204；`/v1/state` 中 `s1 = needs_input`，`question = "可以删除 temp 目录吗?"`，顶层 `mood` 由 `calm` 翻转为 `alert`。

Step 4 · 全状态机过一遍 (第二会话 + 失败 + 退役)

目的: 验证排序与老化规则。操作:

```
curl -s -X POST http://127.0.0.1:2799/v1/hook/session-start -d '{"session_id\":"s2\","cwd\":"F:/reminder/vault\"}'
curl -s -X POST http://127.0.0.1:2799/v1/hook/subagent-start -d '{"session_id\":"s2\"}'
curl -s -X POST http://127.0.0.1:2799/v1/hook/stop-failure -d '{"session_id\":"s2\"}'
curl -s -X POST http://127.0.0.1:2799/v1/hook/stop -d '{"session_id\":"s1\"}'
curl -s http://127.0.0.1:2799/v1/state
curl -s -X POST http://127.0.0.1:2799/v1/hook/session-end -d '{"session_id\":"s2\"}'
```

✅ **预期结果:** s2 = **failed** (question="API error", subagents=1, 排序在 working/done 之前) ; s1 = **done**; session-end 后 s2 消失 (gone 即退役), totals 同步收敛。

Step 5 · 打开面板, 无头浏览器自证

目的: 验证 /panel/ 页面轮询 /v1/state 并画出"螃蟹心情"。操作:

```
msedge --headless --disable-gpu --window-size=900,500 --screenshot="panel_screenshot.png"
"http://127.0.0.1:2799/panel/"
```

✅ **预期结果:** 真实截图 panel_screenshot.png (见下图): 🦀⚠️ "有会话在等你" + 橙色 needs_input 卡片 + 中文问题 + "燃烧 440 tok (4 行)"。



有会话在等你

clawdeck — needs_input: 可以删除 temp 目录吗? · 燃烧 440 tok (4 行)

加分项 · 实证上游 fail-open

直接跑仓库自带的 `hooks\sidecrab_statusline.py`，stdin 喂假 statusline 文档，此时 crabd 并未运行：

```
echo '{"session_id":"test","model":{"id":"claude-opus-5-5","display_name":"Opus"},
      "workspace":{"current_dir":"F:/reminder/clawdeck"}}' | python
..\repo\hooks\sidecrab_statusline.py
# → 🌸 sidecrab · Opus · clawdeck 退出码 0
```

✅ **预期结果：**无崩溃、无堆栈，静默降级为最小状态行——“仪表盘失联不连累会话”由假设变成实测。

踩坑记录：① Git Bash 命令行内联中文 JSON 会以错误编码发出（curl 报 000），改用 `--data-binary @utf8.json` 稳妥；② 上游 crabd 声明 Python 3.13，本机 3.14 亦可运行实验件（mini-crabd 仅用标准库，无关版本差异）。

知识库关联：本篇新立概念 [ClaudeCodeHooks与fail-open](#) · [环境显示与触屏Agent面板](#) · [单一馈送与schema冻结](#)
关联已有：[[人机协同Interrupt]] · [[JSONL事件日志与折叠模型]] · [[零依赖编程]] · [[健康探测三态]] · [[提示注入]] · [[发布交换与健康门回滚]]
项目笔记：[[项目笔记/clawdeck]] · 源仓库：github.com/Dixie-sketch/Clawdeck（MIT） · 本地克隆：
F:\reminder\clawdeck\repo